



DEVFEST MILANO 2026 · 10 OTTOBRE

L'incidente non parla PromQL

Un'interfaccia naturale sopra il tuo stack di osservabilità

Francesco Montelli · Software Engineer freelance

`montelli.dev`



`montelli.dev/talks/incidente-non-parla-promql`

IL PROBLEMA



**«Dimmi cosa è
successo.»**

Quattro atti

01

Tre segnali, una lingua

Cosa c'è sotto: log, metriche, trace e OpenTelemetry.

02

Dalla domanda all'agente

Cervello, mani, occhi: un pezzo alla volta.

03

La squadra

Più agenti, un coordinatore, e chi decide cosa.

04

Coordinare

Più agenti insieme: cosa cambia, quanto costa.

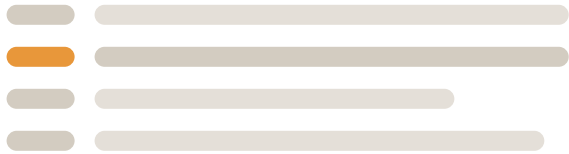
Atto I

Tre segnali, una lingua

Cosa vede il sistema quando qualcosa si rompe?

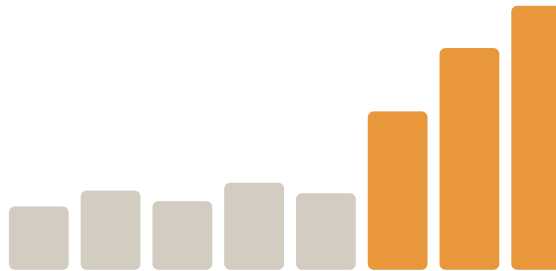
Tre segnali, tre domande diverse

LOG



Cosa è successo, in un punto.
Non come ci sei arrivato.

METRICHE



Qualcosa è cambiato.
Non perché.

TRACE



Dove si è perso il tempo.
Su una richiesta sola.

Raccoglie, standardizza, instrada

1 · raccoglie

2 · standardizza

3 · instrada



Atto II

Dalla domanda all'agente

E se a parlarci fosse un modello?

La cosa che si prova per prima



Dati veri, domanda sbagliata

nel prompt, oltre alla domanda		tornano dati	risposta giusta
1	niente i nomi delle metriche li inventa	0,7 / 20	0,7 / 20
2	l'elenco dei nomi veri store normalizer traces_spanmetrics_latency_bucket ...	8,7 / 20	2,0 / 20

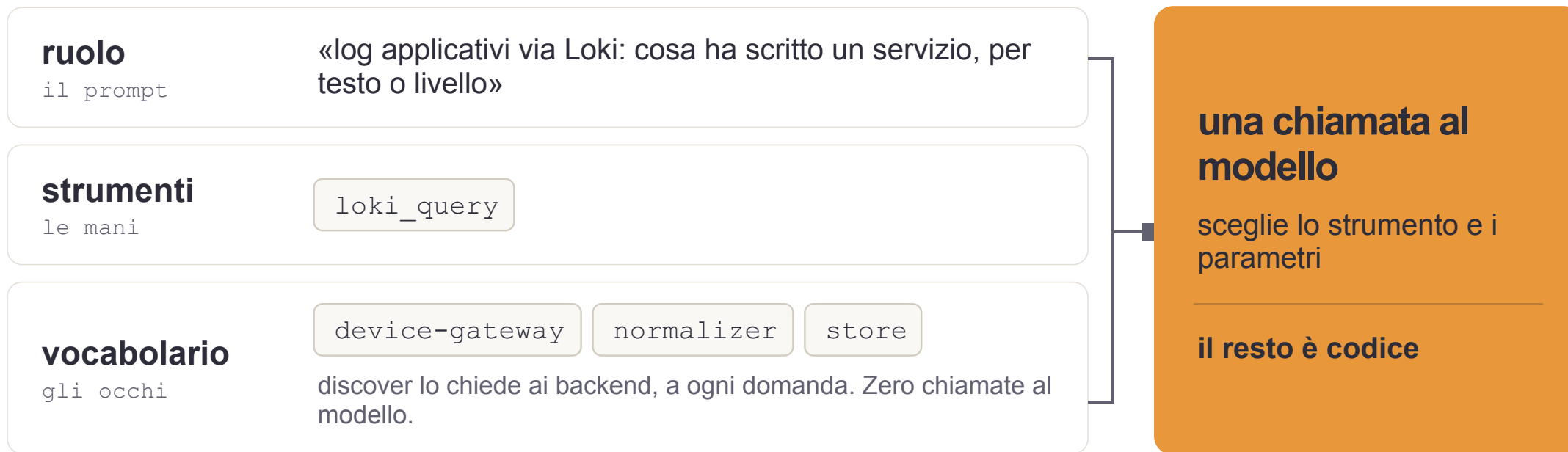
Le query smettono di essere vuote. **Le risposte restano sbagliate.**

20 domande · cambia solo il prompt · media di 3 esecuzioni

Dallo scrivere allo scegliere



Un ruolo, i suoi strumenti, il suo vocabolario



nella demo · 3 servizi · 221 metriche · 54 pannelli

Atto III

La squadra

Un guasto non fa una domanda sola.

Perché non un agente solo?

istruzioni per ogni ruolo

schemi di tutti gli strumenti

risultati dei giri precedenti

la domanda

- 1 Troppe opzioni insieme**
più strumenti, più facile scegliere quello sbagliato
- 2 Context rot**
più il contesto cresce, peggio il modello lo usa
- 3 Un prompt per tutti i ruoli**
istruzioni generiche, o in conflitto fra loro

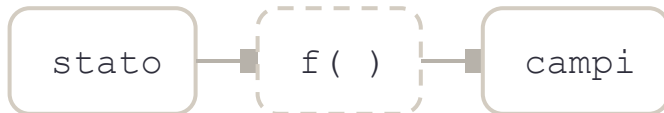
Quando indago, mi faccio quattro domande

«I dati non arrivano più sul DB.»



Nodi e archi

Nodo



dentro, può esserci

solo codice

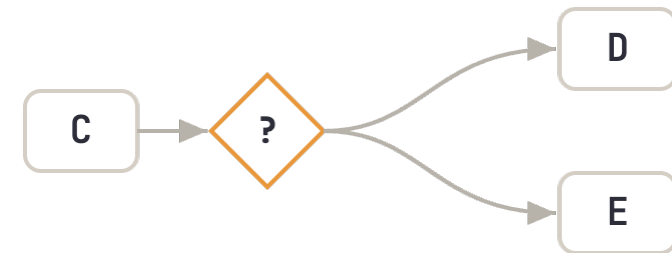
una chiamata al modello

un agente

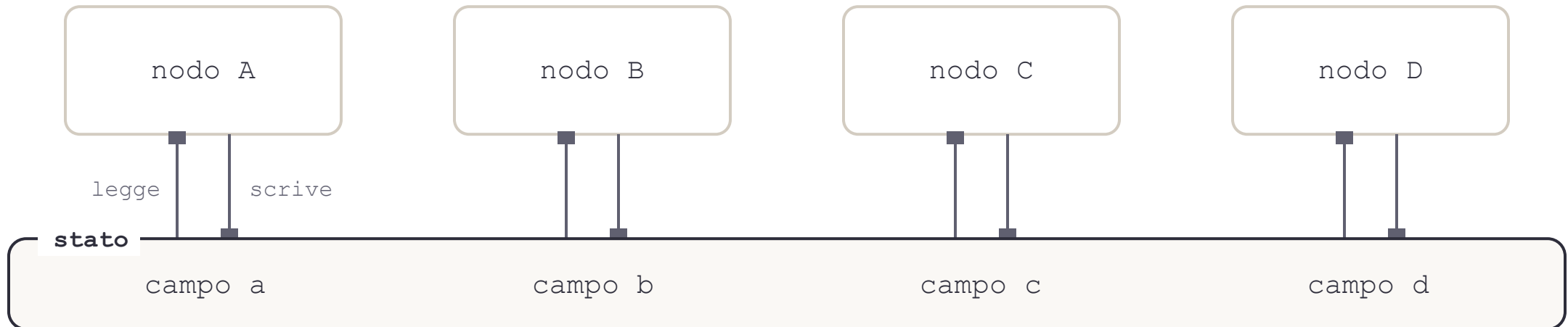
Arco



Arco condizionale



Lo stato: la memoria condivisa



I quattro pezzi, in un grafo



Quando fermarsi lo decide il codice



Un giro: un'ipotesi, un agente



Una causa confermata. Le altre restano aperte.

1	store è fermo	confermata
0,8	store è fermo, detto in un altro modo	ripete la prima
0,5	normalizer ha un errore	aperta · non indagata
0,5	device-gateway ha un errore	aperta · non indagata

Un sistema che non può dire «non lo so» dirà qualcos'altro

```
$ make chiedi Q=quanti utenti hanno abbandonato il carrello oggi?
```

```
Risposta: Non ci sono dati sufficienti per rispondere alla domanda.
```

stesso sistema, stessa domanda: 8 esecuzioni · la risposta qui sopra è una delle 6



6 · «non lo so»

1 · inventata 1 · errore

Ma una volta su otto, lo stesso sistema: **556.300 utenti**, con confidence 0,9.

Dove siamo arrivati

2_{/20}

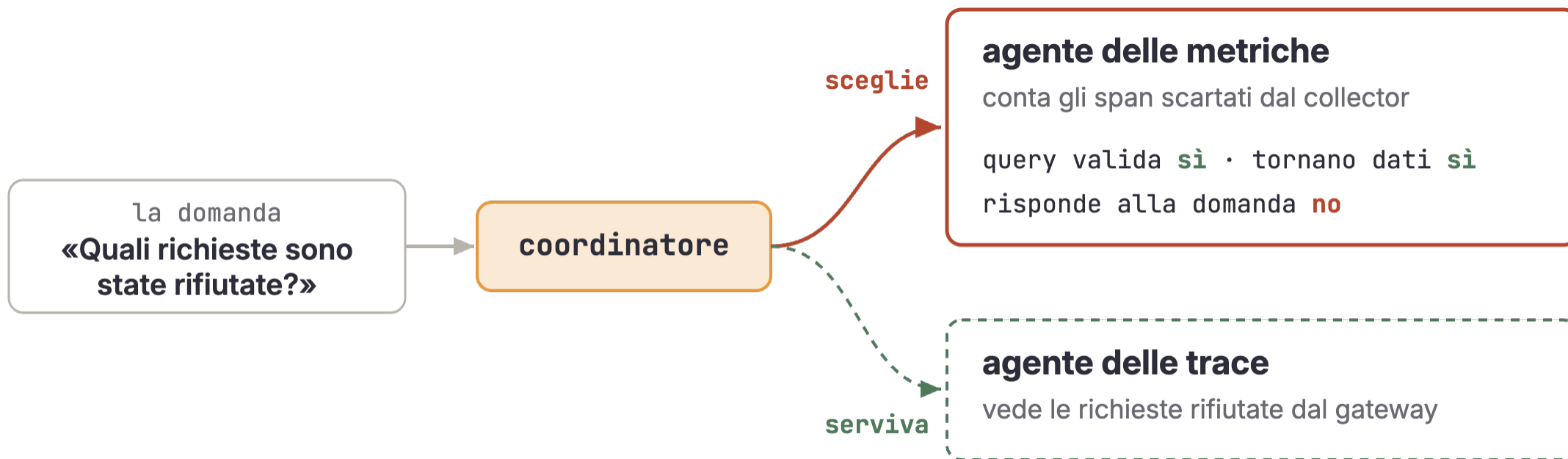


11_{/20}

il modello scrive

l'agente sceglie, il codice compone

La query è giusta. Il posto no.



Atto IV

Coordinare

Più agenti, e le regole che li guidano.

E se il primo indizio fosse il sintomo?



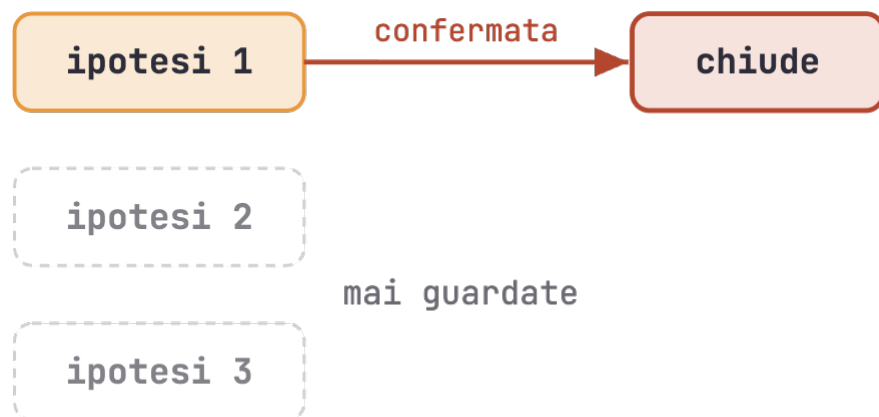
Chiude alla prima che regge. **Non confronta.**

Il grafo di adesso



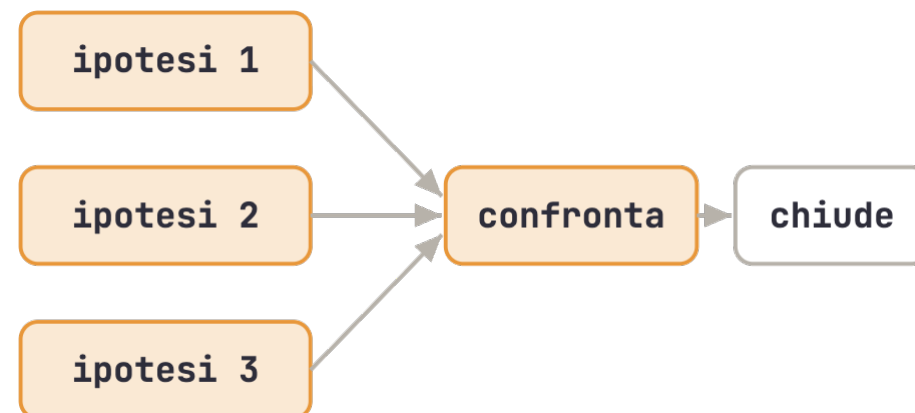
Il primo che regge, o il migliore fra tutti

adesso · il coordinatore



Chiude alla prima che regge

quello che provo · la squadra



Le guarda tutte, poi sceglie

Un agente per ipotesi, tutti insieme



Send

un invio per ipotesi, partono insieme

Reducer

i risultati si accodano, nessuno cancella gli altri

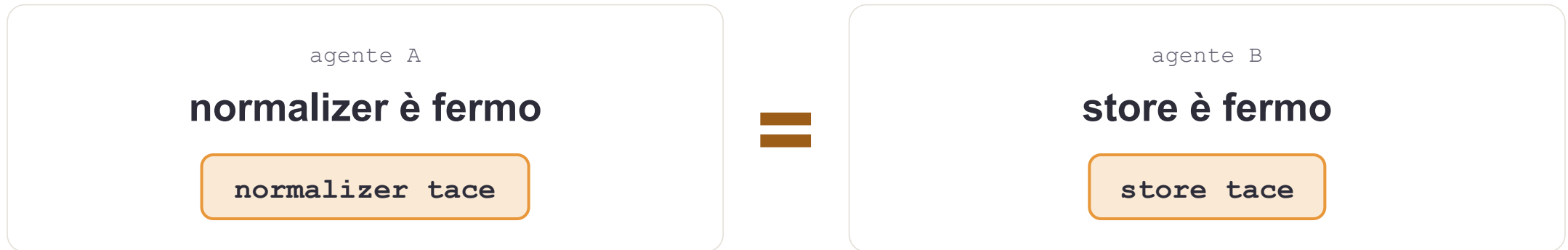
Il suo ciclo

ogni agente indaga da solo, con uno stato privato

Un nodo che decide anche dove andare

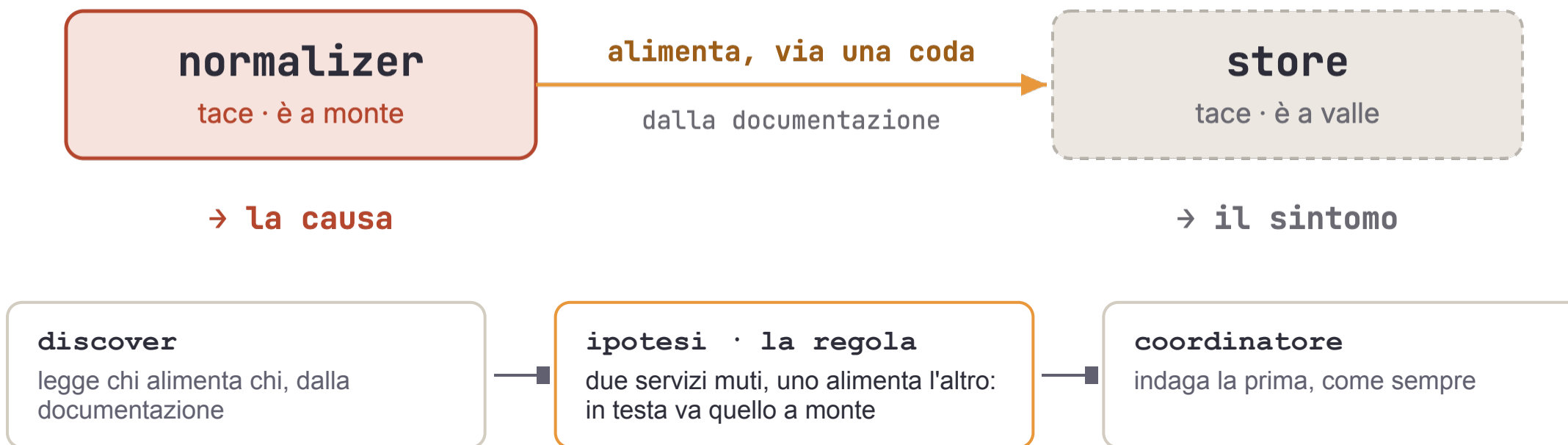


La squadra confronta, ma le prove sono uguali



7_{/33} la stessa prova non separa: **si sceglie a caso**

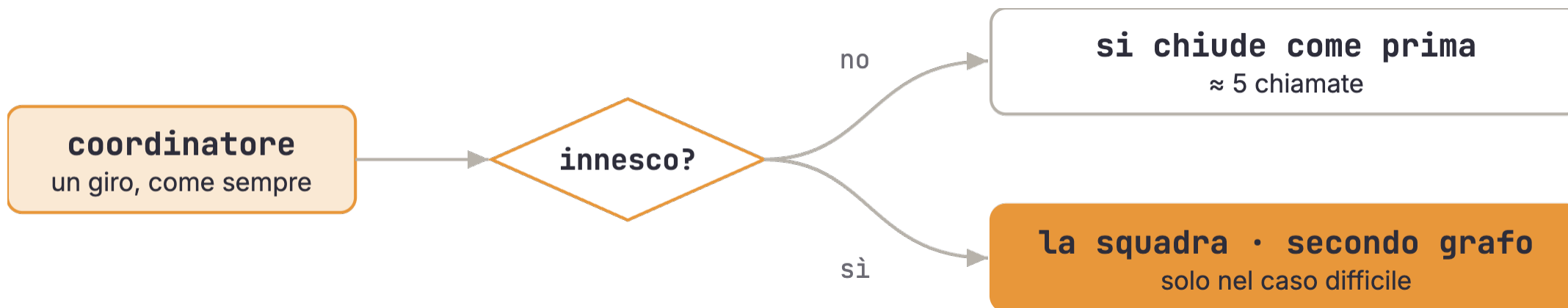
Chi viene prima, scritto nel codice



33/33

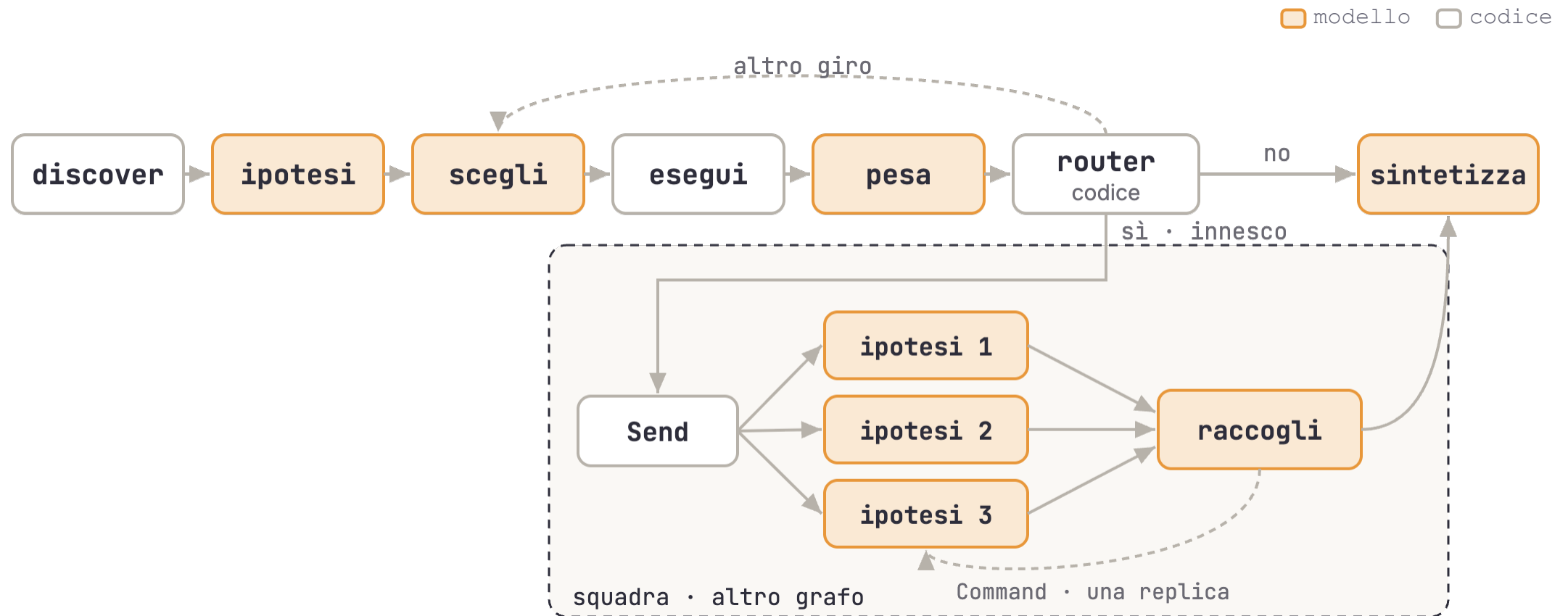
normalizer fermo · 5,2 chiamate per indagine, come prima

La squadra, quando la regola non basta



l'**innesco** due ipotesi sopra 0,7, oppure una catena di servizi muti. **Lo decide il codice.**

La nuova coordinazione, intera



Da zero a trentatré

0_{/33}

il primo che regge
5 chiamate



7_{/33}

la squadra, sempre
15 chiamate



33_{/33}

il coordinatore + la regola
5,2 chiamate

normalizer fermo · 33 indagini per variante · la squadra dietro innesco: 31/33 a 14 chiamate

L'incidente

Un guasto vero

Il sistema finale, su un sistema che si rompe.

Un servizio fermo, due segnali

chi è di turno

«I dati non arrivano più su MongoDB. Cosa sta succedendo?»

i log

store non dice più niente

spento, o solo senza lavoro?

le trace

normalizer gli scrive ancora

su telemetry.clean, adesso

Ogni segnale da solo non basta. **L'agente ha solo la domanda.**

Chiedo in italiano, risponde in italiano

io

«i dati di telemetria non arrivano piu' su MongoDB, cosa sta succedendo?»

il sistema

store e' fermo: non emette segnale, ma normalizer continua a inviargli dati su telemetry.clean.

confidence 1

fonte: la documentazione + loki

2 ipotesi restano aperte

dietro: un agente (loki) · una query composta dal codice · un silenzio documentato

Un agente che scrive il post-mortem

Scegliere o scrivere? **Scrivere.**

Non esiste un enum di post-mortem legali. E da qui cambia tutto il resto.

Più agenti? **No.**

Un vocabolario solo, un agente solo.

La misura cambia natura

Non esiste una gold da confrontare: serve un giudizio umano.

L'affidabilità sta in quello che il modello **non ha il permesso di fare.**

Scrivere la query

→ `la compone il codice`

Inventare un nome

→ `l'elenco viene dai backend`

Chiudere troppo presto

→ `lo decide il router`

Grazie.

Domande?

`pagina del talk · codice, slide, numeri github.com/monte97`



Francesco Montelli

`francesco@montelli.dev · montelli.dev · +39
338 208 1124`



`montelli.dev/talks/incidente-non-parla-promql`